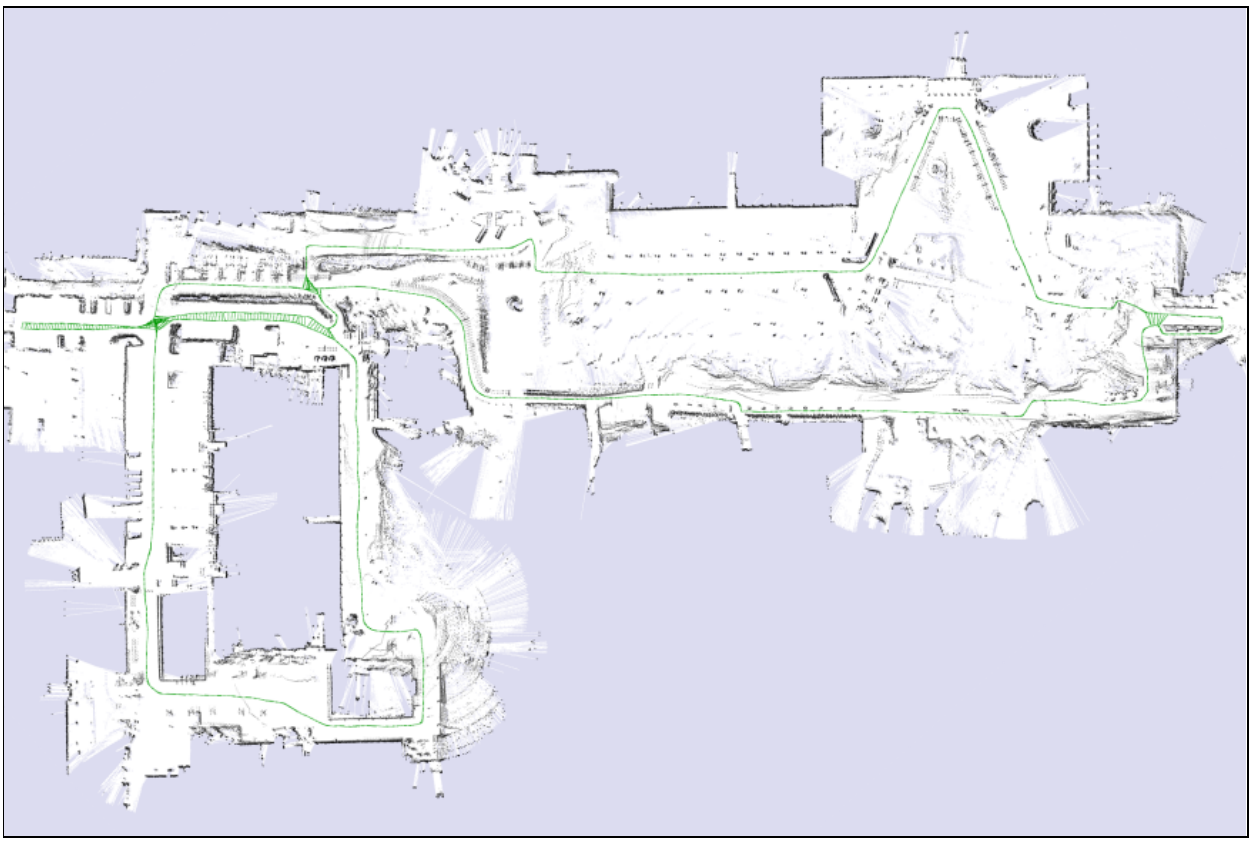


Parallelization Of Localization and Planning Algorithms

GPU Accelerated Particle Filter SLAM

—

Project Supervisor: Dr Mangal Kothari
Department Of Aerospace Engineering
Submitted By: Nitik Jain (Roll No. 170448)
Department Of Mechanical Engineering

Table Of Contents

Certificate From Supervisor

This is to certify that the project entitled “Parallelization of Localization and Path Planning Algorithms (GPU Accelerated Particle Filter SLAM)” submitted by Nitik Jain (170448) as a Undergraduate Project for AE472 during the Semester-I of Academic Year 2019-20, is a bonafide record of the work done by her under my guidance and supervision at the Indian Institute of Technology, Kanpur from August, 2019 to November, 2019.

Dr Mangal Kothari

Assistant Professor,

Department of Aerospace Engineering,

Indian Institute of Technology, Kanpur-208016

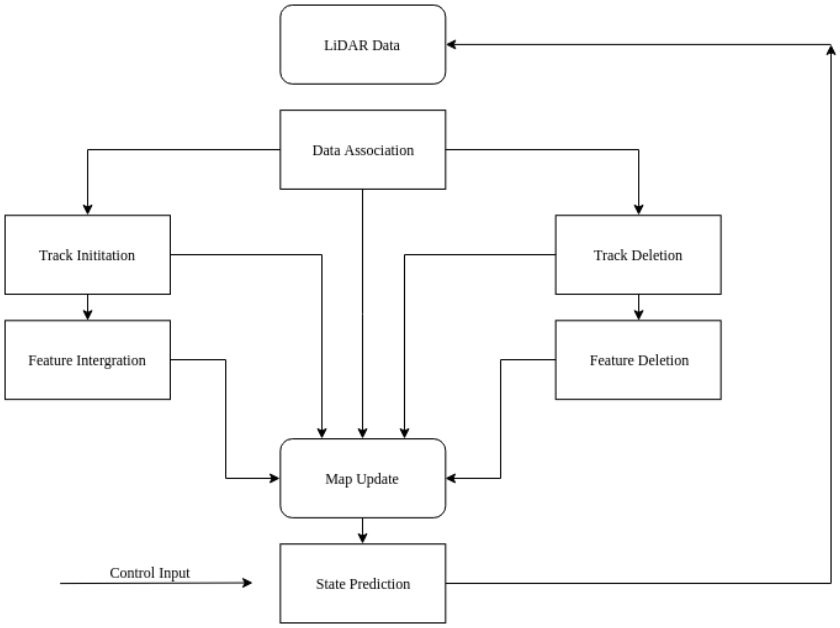
Introduction

Mapping of the environment using sensor data is an important problem in robotics and even more in autonomous vehicles such as cars and quadcopters. The fusion of this data helps us to build a spacial model of the physical environment around the vehicle and also to know its position in the world if the spacial model is already available. This information is very crucial for making decisions while planning trajectories to the destination. In the case of autonomous cars which typically travel at 60kmph, it is crucial to update its position and map the surroundings at a very high rate. This project aims at speeding up one such mapping algorithm called Simultaneous Localization and Mapping(SLAM)[1] using Monte Carlo Localization utilizing the parallelism of the GPU.

The project will be split into two parts.

• Particle Filter[2]: The first part involves building a particle filter. This algorithm helps to localize the car in the given map of the environment. To achieve this, the algorithm starts with a uniform random distribution of particles. Then it uses the sensor information to determine which particle represents the best estimate of the attitude of the vehicle. As the vehicle moves, the estimate of how well each particle performs is tracked.

• SLAM: Particle filter algorithm can be further extended to simultaneously localize & build the map as the car moves through an unknown environment using an occupancy grid[3].



Overview of SLAM Algorithm for Mapping and Localization

Literature Review and Motivation for Project

Particle Filters One of the biggest challenges in robotics operating in social environments is the task of building a map of the region and accurately identifying where the robot is (known as Simultaneous Localization and Mapping, or SLAM). Since the late ‘90s, one of the most successful algorithms for solving this problem is using a particle filter to estimate the robot position relative to its observations, and build the map successively as new measurements are made. A particle filter in its simplest form is a Monte Carlo estimation of the current robot state. By creating a large number of potential robot positions and checking each one relative to past knowledge, accurate estimates of true coordinates can be achieved without complex regression fitting of sensor data.

The biggest limitation of particle filters is that for a large number of particles, CPU implementation is incredibly costly. To compensate for this, historically SLAM algorithms relied heavily on good robot odometry to create a prior on the distribution using dead-reckoning. However, as mobile graphics processors become more viable for robotic applications increasing the particle count becomes a great way to improve estimation accuracy without significant runtime increase. In fact, the particle filter algorithm for SLAM can be considered embarrassingly parallel in implementation, and accurate results can be achieved from visual sensor data only, without any need to develop a prior with robot odometry.

An Overview of Particle Filter

The idea of the particle filter (PF: Particle Filter) is based on Monte Carlo methods, which use particle sets to represent probabilities and can be used in any form of the state-space model. The core idea is to express its distribution by extracting random state particles from the posterior probability. It is a sequential importance sampling method (Sequential Importance Sampling).

Particle filters methods are recursive Bayesian filters which provide a convenient and attractive approach to approximate the posterior distributions when the model is nonlinear and when the noises are not Gaussian.

Algorithmic flow for Particle Filter

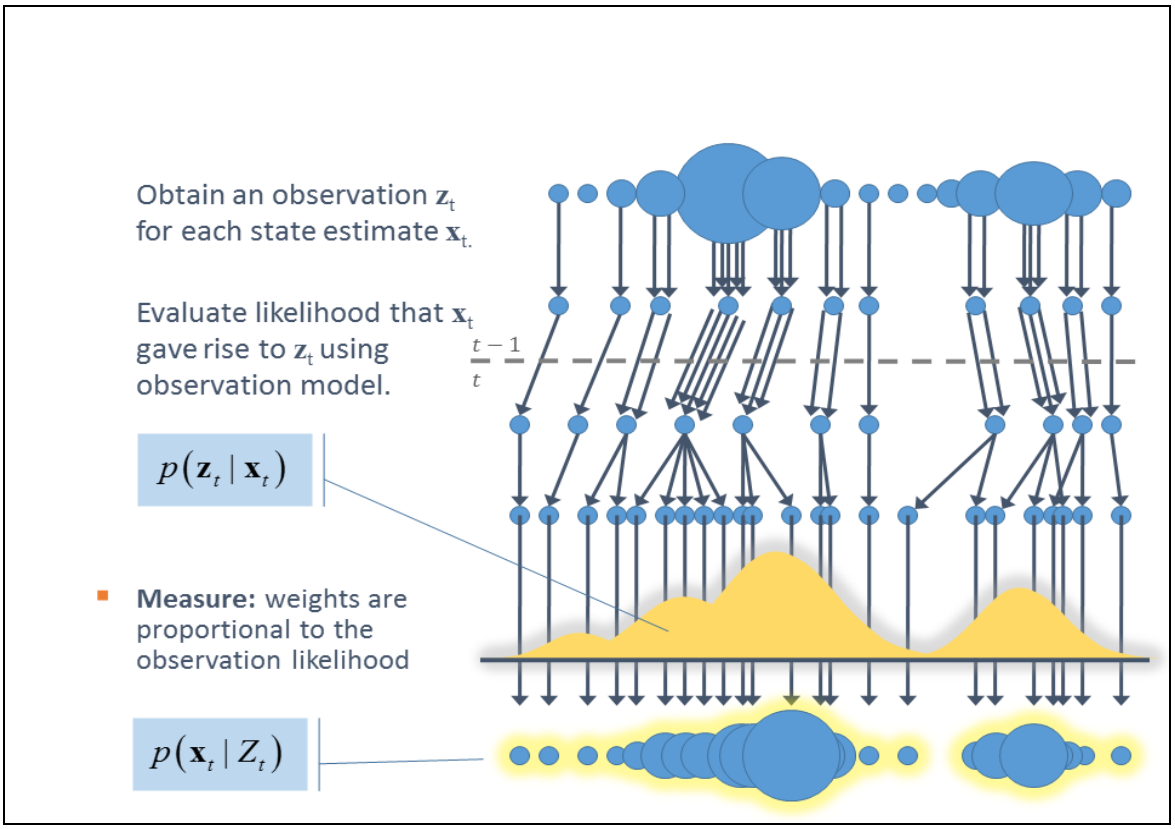
1. Algorithm Particle_filter(X_{t-1}, u_t, z_t):
2. $X'_t = X_t = \emptyset$
3. for $m = 1$ to M do
4. sample $x_t^{[m]} \sim p(x_t \mid u_t, x_{t-1}^{[m]})$
5. $w_t^{[m]} = p(z_t \mid x^{[m]})$
6. $X'_t = X'_t + \langle x_t^{[m]}, w_t^{[m]} \rangle$
7. endfor
8. for $m = 1$ to M do
9. draw i with probability $\propto w_t^{[i]}$
10. Add $x_t^{[i]}$ to X_t
11. endfor
12. return X_t

Step 4: Each particle is a hypothesis as to what the true world state may be at time t and sampling from the state transition distribution

Step 5: Importance Factor, incorporates the measurement into particle set

Step 8-11: The loop for Re-sampling the importance of sampling

Flow chart representing the algorithm



In simple terms, the particle filtering method refers to the process of obtaining the state minimum variance distribution by finding a set of random samples propagating in the state space to approximate the probability density function and replacing the integral operation with the sample mean.

The sample here refers to the particle, and when the number of samples $N \rightarrow \propto$, it can approximate any form of probability density distribution.

Although the probability distribution in the algorithm is only an approximation of the real distribution, due to the non-parametric characteristics, it can get rid of the constraint that the random quantity must satisfy the Gaussian distribution when solving the nonlinear filtering problem, and can express a wider distribution than the Gaussian model.

It also has a stronger ability to model the nonlinear characteristics of variable parameters. Therefore, particle filtering can accurately express the posterior probability distribution based on observation and control quantities, which can be used to solve the SLAM problem.

Markov Chain Monte Carlo improvement strategy[4]

The Markov Chain Monte Carlo (MCMC) method produces samples from the target distribution by constructing Markov chains with good convergence. In each iteration of the SIS, the MCMC is combined to move the particles to different locations to avoid degradation, and the Markov chain can push the particles closer to the probability density function (PDF), making the sample distribution more reasonable.

SLAM: Simultaneous Localization and Mapping:[5]

In navigation, robotic mapping and odometry for virtual reality or augmented reality, simultaneous localization and mapping (SLAM) is the computational problem of constructing or updating a map of an unknown environment while simultaneously keeping track of an agent's location within it. While this initially appears to be a chicken-and-egg problem there are several algorithms known for solving it, at least approximately, in tractable time for certain environments. Popular approximate solution methods include the particle filter, extended Kalman filter, Covariance intersection, and GraphSLAM.

Basic Particle Filter SLAM Algorithm[6]

1. The Particle Filter algorithm can be broken into six basic steps:
2. Disperse all particles with Gaussian random noise.
3. For each particle, calculate which grid cells the sensor detects an obstacle, and sum these values.
4. Select the highest-scoring particle as the new position for the next time step, and adjust all particle weights relative to the distribution of scores calculated in step 2.
5. Update the map by increasing the value of detected collisions, and decreasing the value of cells between the current position and each measured collision.
6. Resample the particles proportionally to their weight distribution.
7. Repeat steps 1 through 5 for each successive sensor input.

SLAM through Particle Filter In Robotics:

Consider the first example where you had to examine the surrounding by your hands. Suppose there are N of you and are randomly spread out in the surrounding and each of you has a distance vector which contains the distances from each of the obstacles. Since there are 1000 of yous, many of them would be in the vicinity of you near the same obstacle. Of course, the orientation may vary. Currently, all human beings are uniformly distributed. Until the next step which is called resampling. In the resampling process, only the humans near you would remain and others will disappear. You are said to be somewhat localized.

Formally, a particle is a discrete guess of where a robot may be located. Also, regularly, these particles are resampled with replacement from the distribution so that the particles consistent with the measurements survive. After successful localization, the particles are collected in a region of the high probability of the robot.

Link to Repository: https://github.com/nitik1998/GPU-Accelerated-SLAM-UGP-III

Objectives

The project will be divided into four stages:

1. Setting up of environment(hardware & software) and sensor configuration
2. CPU and GPU implementation of Particle Filter.
3. GPU implementation of SLAM.
4. GPU vs CPU performance analysis of various stages and the testing on benchmarking dataset

Platform

The hardware to be used for the project is Nvidia Jetson TX1, and the data would be recorded in terms of bag file from sensors (2D LiDAR: Rplidar A3)on the Unicycle model of Nex Robotics Ox Delta robot.[7]

A Lidar sensor and a camera would be used for sensing the environment. The project would be implemented on Robot Operating System(ROS)[8] framework. It provides us abstraction to variation of data formats of different sensors and also allows easy flow of data between other modules of the vehicle(planning and control). C/C++ would be used as the primary language and CUDA for GPU implementation.

Modified Robot and it's CAD model with sensors

Image Source: https://www.slamtec.com/en/Lidar/A3

Implementation Methodology

Building the project:

1. Install Robot Operating System from http://wiki.ros.org/ROS/Installation
2. Create a catkin workspace using http://wiki.ros.org/ROS/Tutorials/CreatingPackage
3. Clone the files from here into the src folder of the created Catkin workspace
4. *catkin_make* from the home of the workspace
5. Source the workspace using

   source devel/setup.bash

6. run *roscore*
7. Run the Lidar node and odometry node
8. Run the GPU slam using

   rosrun slam gpuSLAM

9. To visualize data, run rviz. An rviz vizualization configuration file has been provided.

Process Overview

- Convert scan from polar to Cartesian coordinates (parallelized)
- Update the motion of the particle-based on Odometry
- Generate uniformly distributed particles around odometry position and orient scans based on the new position (parallelized)
- Calculate correlation scores by comparing with the map (parallelized)
- Update weights and obtain pose of the best particle (parallelized)
- Create a local map with the best pose (parallelized)
- Copy the map from the local buffer to global buffer (parallelized)
- Check and Resample particles (parallelized)

Algorithm Overview

The following stages are involved,

a) Initialize a uniform distribution of particles

b) Perform kinematics update of vehicle

c) Calculate the correlation score of particles

d) Update Particle weights

e) Choose best particle and update map

f) Recalibrate weights if needed.

ROS-Nodes Overview

Subscribed Topics

- *scan (sensor_msgs/LaserScan)* - Laser scans from Lidar
- *scanmatch_odom(nav_msgs/Odometry)* - Odometry from dead reckoning with respect to global map

Published Topics

- *lmap (nav_msgs/OccupancyGrid)* - Local map orientied based on best pose
- *map (nav_msgs/OccupancyGrid)* - Global map
- *pose (nav_msgs/Odometry)* - Pose with respect to initial starting position

The following nodes are there in the src folder of the repository.

Tested on: Linux Ubuntu 16.04, Nvidia Tegra T1 @ 2.32GHz 4GB, Intel NUC Mini PC, Processor Intel Core i7 (8th Gen) (NUC8i7BEH) @ 3.3GHz 4GB RAM

Parameters

- *particles* - Number of particles for Particle filter (default:500)
- *motion_noise_x* - Initial pose covariance (x*x), used to initialize filter with uniform distribution. (default:0.05)
- *motion_noise_y* - Initial pose covariance (y*y), used to initialize filter with uniform distribution. (default:0.05)
- *motion_noise_theta* - Initial pose covariance (yaw*yaw), used to initialize filter with uniform distribution.(default:0.0534)
- *lmap_width* - Local map width(default:60)
- *lmap_height* - Local map height(default:60)
- *lmap_resolution* - Local map resolution(default:0.1)
- *gmap_width* - Global map width(default:100)
- *gmap_height* - Global map height(default:100)
- *gmap_resolution* - Global map resolution(default:0.1)
- *Sat_thresh* - saturation threshold of the occupancy grid cells(default:120)
- *neff_thresh* - Resample threshold(default:40)
- *logodd_occ* - log-odd free value(default:3)
- *logodd_free* - log-odd occupied value(default:1)

Results

The following image was built using LIDAR data real-time from the bag file and on benchmarking dataset:

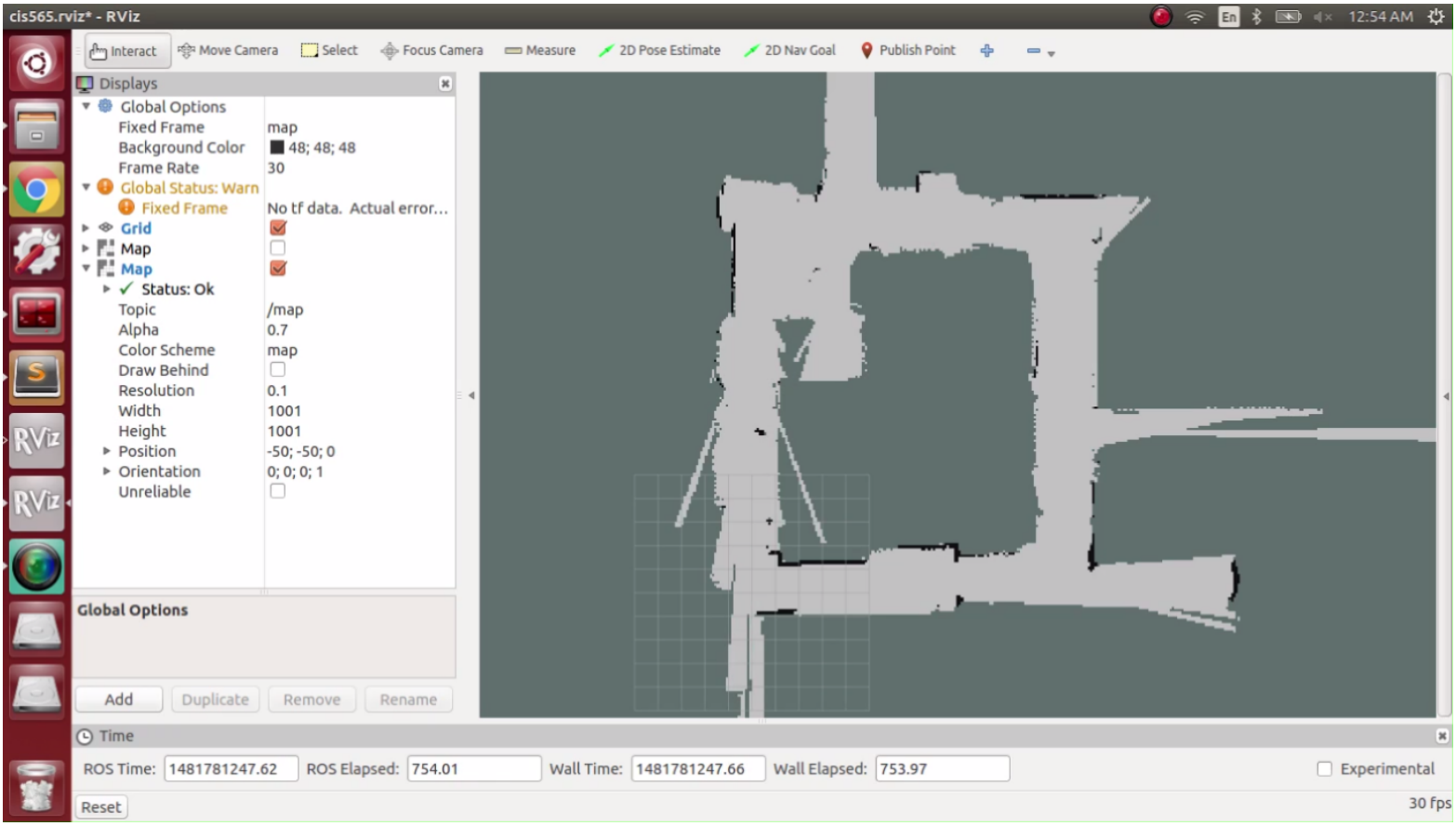
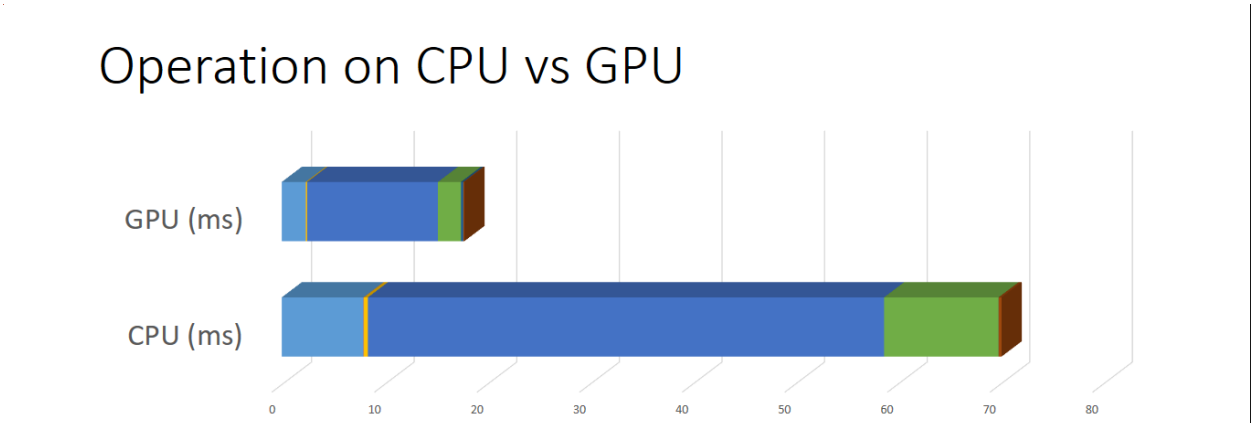


Image showing Rviz Visualization of Map Formed after first loop closure

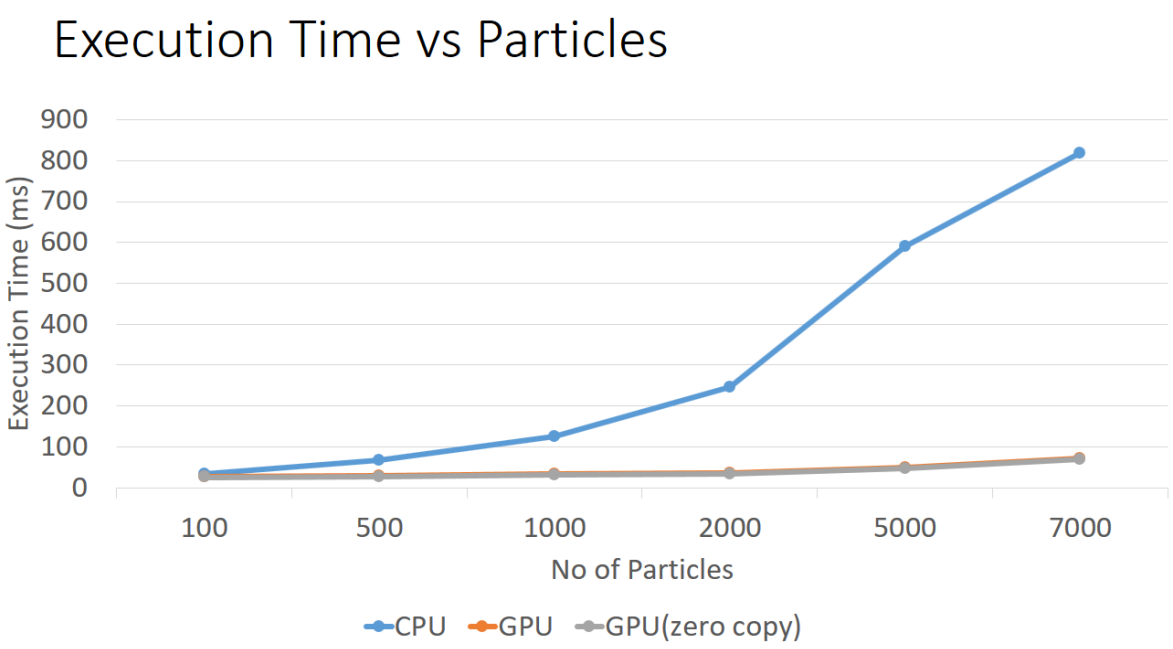
The algorithm was implemented on CPU and GPU. Eight kernels were used to parallelize most parts of the algorithm. The execution time of these sections improved dramatically on the GPU. The different execution times on GPU and CPU for each kernel is shown in the figure below. The number of particles used is 500.



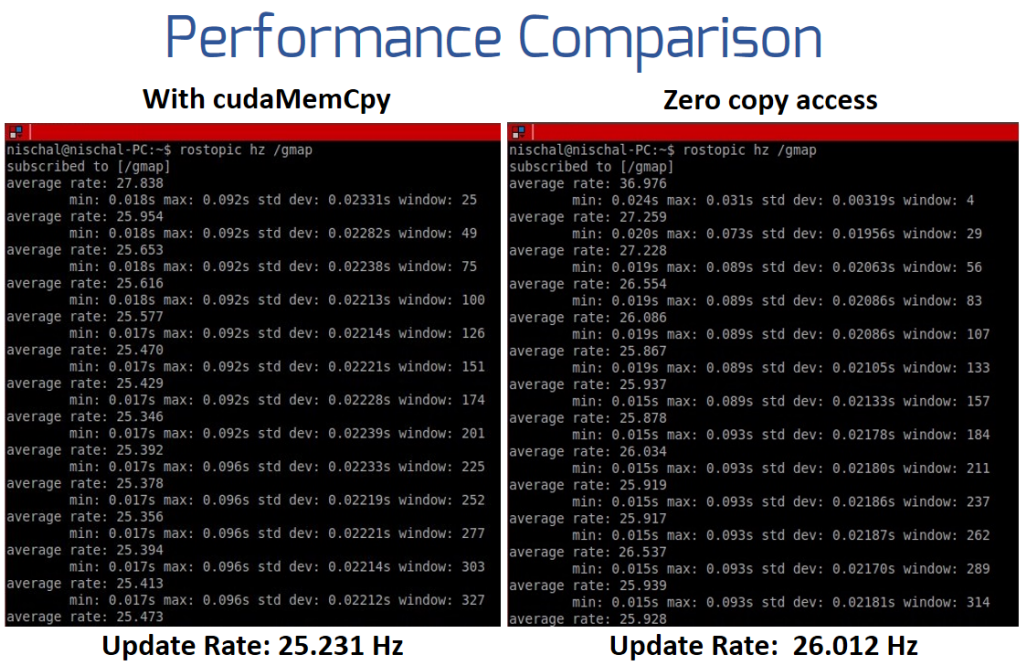
	CPU (ms)	GPU (ms)
copyLocalToGlobal	7.96	2.34
convertToCartesian	0.042	0.007
convertToCells	0.014	0.003
createLmap	0.378	0.128
generateParticles	50.31	12.743
mapCorrelation	11.155	2.243
calculateCDF	0.003	0.241
resampleParticles	0.295	0.098

It is seen that all the functionality had a speedup on the GPU except for the calculation of CDF. This was because the CDF calculation involves an operation of atomic add that defeats the purpose of parallelism.

The total execution time for CPU and GPU for various particle sizes is shown in the figure below. The execution time in the CPU implementation increases exponentially with particle sizes. This is evident because of the nature of series computation. On the other hand, the parallel computation on the GPU has a very linear increase with particle size. Additionally, another implementation with zero-copy was performed as the operations were performed on streaming data. This eliminated the need for copying the data from one CPU to GPU. This implementation on an average executed 1-2ms faster.



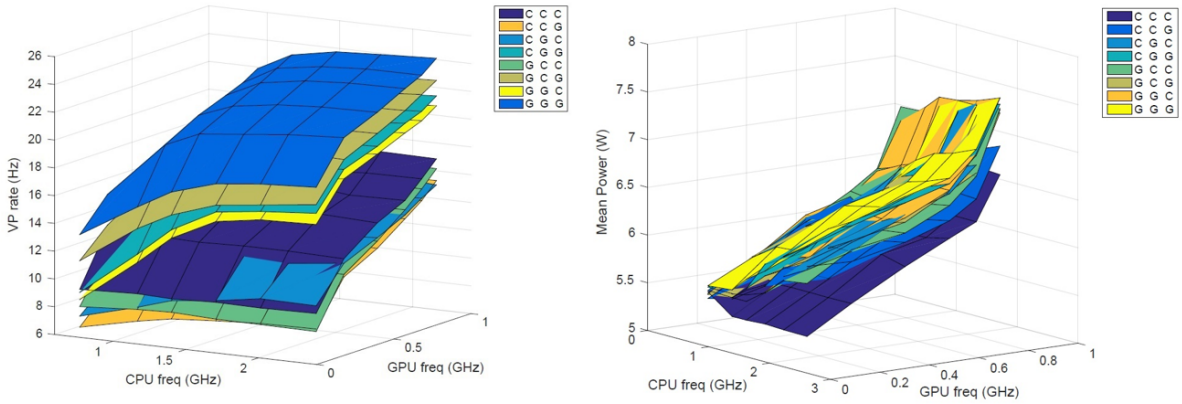
The following figure shows the update rates of GPU implementation with and without zero copy. These are the rates at which the output topics are published to the ROS system.



Another Performance done was on the power consumption of the execution. Three kernels were made to switch dynamically from CPU to GPU (enabled by zero copy)

1. Particle generation and scan orientation
2. Calculation of Correlation Scores
3. Copy of Local Map to Global Map The CPU and the GPU were executed at different clock frequencies with different combinations of the above three kernels on CPU and GPU. The update rates and power consumption were recorded

Operating Frequency



A combination of C C G denotes the first and second operation (mentioned above) run on CPU and the third on GPU. It is seen from the first graph that the update rate is maximum when all the three operations are run on the GPU but at the cost of high power as seen in the second graph. The information from this graph can be used to train a controller to switch to a lower update rate when the map is already known by varying the operations on CPU and GPU and also throttling the frequencies thus saving power.

Conclusion and Future work

It is seen that all the functionality had a speedup on the GPU except for the calculation of CDF. This was because the CDF calculation involves an operation of atomic add that defeats the purpose of parallelism. Thus, using GPU could be beneficial but for systems such as quadcopter and autonomous vehicles, the power consumption graph also comes into play. Thus, the development of a ground station for such a GPU accelerated system could be a viable solution. The work could be integrated with a path planning algorithm such as cc-RRT[9], and a complete motion planning and localization stack could be developed.

References

1. https://en.wikipedia.org/wiki/Simultaneous_localization_and_mapping
2. https://en.wikipedia.org/wiki/Monte_Carlo_localization
3. https://en.wikipedia.org/wiki/Occupancy_grid_mapping
4. Accelerating MCMC Algorithms: https://arxiv.org/pdf/1804.02719.pdf
5. https://en.wikipedia.org/wiki/Simultaneous_localization_and_mapping
6. http://www2.informatik.uni-freiburg.de/~grisetti/pdf/grisetti06jras.pdf
7. http://www.nex-robotics.com/0x-delta/0x-delta.html
8. https://www.ros.org/
9. Chance Constrained RRT for Probabilistic Robustness to Environmental Uncertainty: http://acl.mit.edu/papers/Luders10_GNC.pdf